

Student Name: Rana Omarhan

Student ID: 230103191

Hardware / CPU: 11th Gen Intel Core i5-1135G7, 4 Physical Cores / 8 Logical Threads

Date & Section: 02.10.2026 08-p

Challenge 1

```
D:\University Courses\Parallel Computing\week5>python challenge1.py
Hardware Threads Detected: 8
Threads      | Time (s)      | Speedup      | Efficiency (%)
-----
1            | 3.0223        | 1.00         | 100.0
2            | 1.8525        | 1.63         | 81.6
4            | 0.8935        | 3.38         | 84.6
8            | 0.5616        | 5.38         | 67.3
```

Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
1 Thread (Base)	120,000,000	3.0223	1.00x	100.0%
2 Threads	120,000,000	1.8525	1.63x	81.6%
4 Threads	120,000,000	0.8935	3.38x	84.6%
8 Threads (Max)	120,000,000	0.5616	5.38x	67.3%

B)Numba is a JIT compiler. The first invocation of monte_carlo_pi(10_000) causes the compilation of the Python loop to machine code. Since we do not account for this compilation time, which would inflate the 1-thread baseline and make all other measurements meaningless, the warm-up invocation forces the compilation of the loop to get actual execution time measurements.

C)No. The efficiency has declined from 100% at 1 thread to 81.6% at 2 threads, 84.6% at 4 threads, and 67.3% at 8 threads. The main reasons are (1) contention on memory bandwidth, since all threads read random numbers from one shared memory bus; (2) SMT hyper-threading, where 8 threads run on 4 cores, thus creating pairs sharing one FPU, cache, and execution units; (3) serial component of the program by Amdahl's Law (setup, reduction); (4) possibly contention on the random number generator.

D)Numba recognizes that `inside_circle` is a scalar accumulation variable in the prange loop and automatically creates thread-local accumulation variables that are later merged into one through the tree reduction mechanism. This is the same as OpenMP reduction(`+:inside_circle`). Each thread accumulates into its own private variable and then the last merge is done through the tree reduction mechanism. Thus there

Challenge 2

```
D:\University Courses\Parallel Computing\week5>python challenge2.py
Mandelbrot Rows: 1.7760 s
Mandelbrot Cols: 1.7447 s
```

Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
8 Threads (Max)	2500 x 2500	1.7760	N/A	N/A
8 Threads (Max)	2500 x 2500	1.7447	N/A	N/A

Parallel computation in rows and parallel computation in columns had very similar times (1.7760s versus 1.7447s) with the difference between them being only 1.8 percent. The computation in columns is faster due to a more even distribution of work load in this case – the central part of the Mandelbrot set, which is computationally heavy, being in the middle columns and having the possibility of creating stragglers in row-based distribution of the work load.

Challenge 3

```
D:\University Courses\Parallel Computing\week5>python challenge3.py
Float64 Heat Diffusion: 0.747 s, Throughput: 903.76 Megacells/sec
Float32 Heat Diffusion: 0.365 s, Throughput: 1851.04 Megacells/sec
Runtime reduction factor: 2.05x
```

Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
8 Threads (Max)	1500 x 1500 x 300	0.747 (fp64)	N/A	N/A
8 Threads (Max)	1500 x 1500 x 300	0.365 (fp32)	N/A	N/A

Float64: 903.76 Megacells/sec

Float32: 1851.04 Megacells/sec

B) The runtime decreased from 0.747s to 0.365s (-2.05), and throughput increased from 903.76 to 1851.04 Megacells/sec (+2x). Stencil code is limited by memory bandwidth and not by compute capability. Halving the element size halved the number of bytes transferred between the main memory and cache per time step. Therefore, the bottleneck (memory bus) is improved. ALU is not the limiting factor here; therefore, the impact on computation will be almost zero, while memory throughput becomes twice as high.

C) Stencil code is memory bandwidth-limited. Once all cores together utilize the whole bandwidth of the RAM bus, increasing the number of cores does not lead to an increase of the bandwidth. Each core should load the same set of neighboring cells, and the shared L3 cache and memory controller become a bottleneck. In accordance with Roofline model, operational intensity (FLOPs per byte) of a 5-point stencil is very low, therefore the workloads lie in the memory bounded region of the roofline, where the limit of performance depends on the bandwidth, not peak FLOPs.